

Toward Personalized Differentially Private Learning for Decentralized Local Graphs

Longzhu He , Peng Tang , Chaozhuo Li , Jinhu Fu, Litian Zhang, Li Sun , *Member, IEEE*,
Philip S. Yu , *Life Fellow, IEEE*, and Sen Su 

Abstract—Graph-structured data is increasingly generated and stored in decentralized environments, such as social platforms, mobile applications, and edge networks, where users maintain control over their local graph data. However, collecting and analyzing such decentralized graph data for downstream learning tasks raises significant privacy concerns, as nodes and their attributes often contain sensitive personal information. Local Differential Privacy (LDP) has emerged as a promising solution for privacy-preserving data collection without relying on trusted servers. Nevertheless, existing LDP-based graph learning methods typically assume uniform privacy requirements across users, ignoring the heterogeneous and personalized privacy preferences commonly observed in real-world systems. This uniform treatment leads to inflexible noise injection at the data collection stage, resulting in substantial distortion of graph data and degraded utility in subsequent analysis. To address this limitation, we propose PPGNN, a personalized differentially private framework for decentralized graph data. PPGNN enables user-specific privacy budgets during local perturbation while preserving analytical utility. To handle heterogeneous privacy levels and noise distortion, we design a two-stage solution consisting of a Personalized Perturbation Mechanism (PPM) and a weighted calibration strategy, FlexProp. Extensive experiments on six real-world graph datasets demonstrate that PPGNN effectively balances personalized privacy protection and data utility in decentralized graph learning scenarios.

Received 20 February 2026; revised 6 June 2026; accepted 25 June 2026. Date of publication 7 July 2026; date of current version 8 August 2026. This work was supported in part by the National Key Research and Development Program of China under Grant 2024YFF0907401, in part by the National Natural Science Foundation of China under Grant 62072052, in part by the BUPT Excellent Ph.D. Students Foundation under Grant CX20260030, and in part by the Shandong Provincial Natural Science Foundation under Grant ZR2025MS1038. Recommended for acceptance by S. Shang. (*Corresponding author: Sen Su.*)

Longzhu He, Jinhu Fu, and Li Sun are with the School of Computer Science, Beijing University of Posts and Telecommunications, Beijing 100876, China, and also with the State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications, Beijing 100876, China (e-mail: helongzhu@bupt.edu.cn; fjhu@bupt.edu.cn; lsun@bupt.edu.cn).

Peng Tang is with the School of Cyber Science and Technology, Shandong University, Qingdao 266237, China (e-mail: tangpeng@sdu.edu.cn).

Chaozhuo Li and Litian Zhang are with the School of Cyberspace Security, Beijing University of Posts and Telecommunications, Beijing 100876, China (e-mail: lichaozhuo@bupt.edu.cn; litianzhang@bupt.edu.cn).

Philip S. Yu is with the Department of Computer Science, University of Illinois at Chicago, Chicago, IL 60607 USA (e-mail: psyu@uic.edu).

Sen Su is with the School of Computer Science, Beijing University of Posts and Telecommunications, Beijing 100876, China, also with the State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications, Beijing 100876, China, and also with the Chongqing University of Posts and Telecommunications, Chongqing 400065, China (e-mail: susen@bupt.edu.cn).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TKDE.2026.3710808>, provided by the authors.

Digital Object Identifier 10.1109/TKDE.2026.3710808

Index Terms—Local differential privacy, graph-structured data, graph learning, personalized privacy requirements.

I. INTRODUCTION

GRAPH-STRUCTURED data has become a fundamental data type in modern information systems, widely used to model complex relationships in social networks, mobile applications, and online platforms. In many real-world scenarios, such graph data is decentralized and maintained locally by users, containing sensitive information embedded in node attributes and structural connections. Collecting and aggregating these local graph data for downstream analytical tasks may therefore raise significant privacy concerns [1]. Graph Neural Networks (GNNs) [2], [3] have emerged as a powerful tool for learning from graph data, demonstrating remarkable performance in various downstream tasks such as node classification [4], link prediction [5], and graph classification [6]. However, applying GNNs in decentralized environments requires collecting user data to a central server, which further amplifies privacy risks. This is particularly evident in social mobile applications like Instagram and WhatsApp [7], where servers aim to gather various types of information (e.g., user profiles and interactions) for recommendation purposes. Recent studies have also shown that GNNs are vulnerable to various privacy attacks [1], [8], [9], [10], [11], [12], leading to potential leakage of sensitive information. Therefore, there is an urgent need for innovative privacy-preserving graph learning models.

To mitigate privacy threats, local differential privacy (LDP) [13] has emerged as a pivotal and well-recognized method for safeguarding sensitive data in distributed systems. LDP [13] offers robust privacy guarantees for secure data collection and analysis, and has been widely adopted by major companies such as Google [14], Apple [15], and Microsoft [16]. In the LDP setting, there is a server and multiple user clients, where it is assumed that the server is untrustworthy. Each user ensures privacy by locally perturbing their data (typically through noise injection [17], the magnitude of which depends on the *privacy budget*) before sending it to the server. Given its strong data privacy guarantees, LDP has become a key approach to protecting decentralized local graph node features, which are often sensitive. Recent research has delved into graph learning under LDP for enhanced privacy preservation, including advanced LDP mechanisms, such as the multi-bit mechanism [18], [19], [20], as well as classical LDP mechanisms like the Laplace

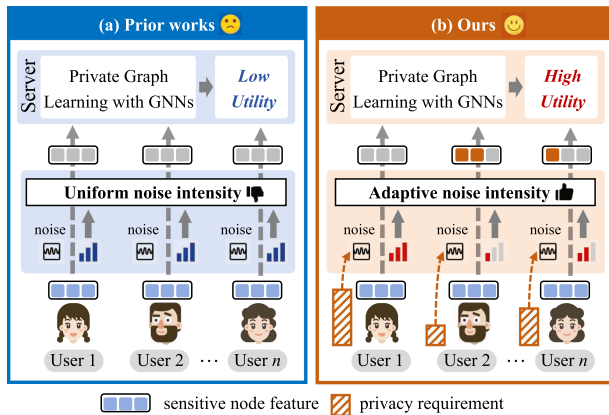


Fig. 1. Comparison of (a) prior works and (b) ours in the locally private graph learning scenario. The scenario involves a server (e.g., a social network server) and multiple decentralized users, each with sensitive node features and distinct privacy requirements. The server collects perturbed data and performs private graph learning using a GNN. (a) Previous works overlook personalized privacy requirements of users and apply uniform noise intensity to protect node features, leading to low utility in private graph learning. (b) In contrast, our approach adapts the noise intensity according to user-specific privacy requirements, thereby promoting high utility in private graph learning.

mechanism [13], Gaussian mechanism [21], and 1-bit mechanism [16]. Fig. 1 illustrates the local private graph learning scenario, where each user has a sensitive node feature. The server collects the perturbed data and performs private graph learning on the noisy data.

However, as shown in Fig. 1(a), prior studies [18], [19], [20], [22], [23] largely overlook the fact that users may have different overall levels of privacy requirement. In real-world scenarios, privacy expectations can vary significantly across users due to factors such as gender, geography, and age. For instance, research [24] shows that females tend to express higher privacy concerns than males, revealing a “privacy gender gap” in the context of online tracking [25]. Similarly, individuals in the U.K. are reported to be less likely to disclose negative emotions than those in Germany or France, suggesting a “privacy geography gap.” These findings indicate that certain user groups may prefer stronger privacy protection across all of their personal data. Despite this, existing locally differentially private graph learning methods assume a uniform privacy level for all users. As shown in Fig. 1(a), this leads to the application of a global maximum noise intensity determined by the most privacy-sensitive user to all node features. Such a one-size-fits-all strategy significantly degrades model utility, as excessive noise added to satisfy a small subset of users unnecessarily undermines the effectiveness of the overall learning process.

To address the above issue, as illustrated in Fig. 1(b), we propose a novel privacy-preserving graph learning framework grounded in the notion of *personalized* LDP (PLDP) [26]. This framework adaptively adjusts noise intensity to meet users’ privacy requirements while enhancing the utility of private graph learning. PLDP, an extension of the traditional LDP, is designed to offer varying levels of privacy protection tailored to individual users or data points. Introducing a personalized definition to LDP enables dynamic adjustment of privacy parameters based on factors such as importance, sensitivity, or specific user requirements.

However, under this personalized privacy setting, two key challenges arise. On the one hand, dual protection of privacy elements becomes essential: not only must node features be safeguarded, but the privacy level [27] of each user also requires protection. A user’s privacy level reflects their valuation of privacy, which can inadvertently reveal sensitive personal attributes that necessitate additional safeguarding. Protecting a user’s privacy level necessitates designing a specialized LDP protection algorithm and addressing the interplay between node feature protection and privacy levels. Improper handling of this interaction risks disclosing the user’s privacy preferences, making the synergistic protection of both aspects a critical consideration. In this dual privacy setting, rationally allocating the privacy budget between the two elements to maximize the utility of privacy-preserving graph learning is vital. On the other hand, noise calibration in multilevel privacy scenarios poses additional challenges. Directly training GNNs with perturbed features can severely reduce the utility of privacy graph learning due to excessive noise. While previous works attempt denoising via multi-hop aggregation [28], [29], [30], this approach proves suboptimal in a multilevel privacy setting because perturbation levels differ for each node. Simple multi-layer aggregation fails to exploit the effective information from each node fully. Consequently, achieving accurate noise calibration under a personalized privacy framework introduces greater complexity than non-personalized approaches.

In this paper, we propose a novel locally differentially private graph learning framework, PPGNN (Personalized Privacy-preserving Graph Neural Network), which integrates two core components: the Personalized Perturbation Mechanism (PPM) and the FlexProp algorithm. Specifically, PPM is designed to achieve dual privacy protection for node features and privacy levels. It comprises two fundamental elements: the Multi-dimensional Local Randomizer (MLR) and the Extended Square Wave (ESW) mechanism. The MLR protects multi-dimensional node features and ensures their utility through dimensionality reduction and rigorous theoretical considerations. Meanwhile, the ESW mechanism enhances privacy protection by extending the capabilities of the traditional square wave mechanism [31] to cover users’ privacy levels across discrete domains, ensuring that the protection is effective and adaptable to varying user requirements. To tackle the challenges of calibrating the perturbed node features, we introduce a weighted aggregation algorithm called FlexProp. FlexProp is essential for integrating individual users’ privacy levels and aggregating information from neighboring nodes within K hops. By applying varying weights based on privacy levels, FlexProp facilitates nuanced denoising, thus improving data quality while maintaining robust privacy measures. This aggregation process supports privacy-preserving graph learning, ensuring the GNN performs effectively despite the added noise. Notably, our PPGNN framework is designed to be compatible with various GNN architectures, providing both flexibility and adaptability for different application scenarios. Our contributions are summarized as follows:

- *Problem:* We propose a novel and realistic problem in differentially private GNNs, aiming to achieve personalized privacy preservation for users’ local data.

TABLE I
NOTATIONS

Symbol	Description
$ \mathcal{V} $	the number of users
$\mathbf{A}$	the adjacency matrix
$\mathbf{X}$	the node feature matrix
τ	the privacy level of users
d	the number of feature dimensions
m	the number of sampled dimensions
ϵ	the privacy budget
$\mathcal{M}$	the perturbation mechanism
$\mathbf{x}_v$	the original feature vector of user v
$\mathbf{x}'_v$	the perturbed feature vector of user v
$\mathcal{N}(v)$	the set of neighbors of v (including v itself)
$\mathbf{h}_v$	the original embedding of node v
$\hat{\mathbf{h}}_v$	the estimated embedding of node v
γ	the privacy budget allocation ratio between PPM and FlexProp modules
w_{ij}	the aggregation weight from node j to node i in weighted message passing
g	the user's true privacy level index (input to ESW)
g'	the randomized output of ESW
f	the size of the input domain in ESW
b	the smoothing window radius in ESW

- *Methodology*: We introduce PPGNN, a novel locally differentially private GNN model designed to meet users' personalized privacy requirements while ensuring the utility of privacy-preserving graph learning.
- *Experiments*: We conduct extensive experiments on six real-world datasets and three GNN architectures (GCN, GraphSAGE, GAT), demonstrating that PPGNN consistently outperforms strong baselines and achieves utility close to the non-private upper bound in multiple settings.

Organization: The rest of this paper is organized as follows. Section II introduces background knowledge and formulates the problem. Section III describes our personalized privacy-preserving graph learning framework, PPGNN. Section IV presents extensive experimental results. Section V reviews the related literature, and Section VI concludes the paper.

II. PRELIMINARIES

In this section, we first provide a formal definition of the problem (Section II-A). Next, we present essential background knowledge related to graph neural networks (Section II-B) and local differential privacy (Section II-C), followed by an introduction to the four primary node feature LDP mechanisms (Section II-D). We summarize the important notations of our paper in Table I.

A. Problem Definition

We provide a formal definition for the problem of learning a GNN with node data privacy. Consider a graph $\mathcal{G} = (\mathcal{V}, \mathbf{A}, \mathbf{X})$, where $\mathcal{V} = \{v_1, v_2, \dots, v_{|\mathcal{V}|}\}$ is the node/user¹ set, and $|\mathcal{V}|$ represents the total number of users within the network. Each user v_i holds locally a privacy feature vector $\mathbf{x}_i \in \mathbb{R}^d$, where d denotes the dimension of the node feature vector. In graph

$\mathcal{G}$, the adjacency matrix $\mathbf{A} \in \mathbb{R}^{|\mathcal{V}| \times |\mathcal{V}|}$ encodes the connections between nodes, while the node feature matrix $\mathbf{X}$ is defined as $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_{|\mathcal{V}|}\}$. Consistent with prior works [18], [19], [20], [22], we focus on the node classification task, which serves as a fundamental building block for various applications in social networks and beyond. Specifically, given a labeled node set $\mathcal{V}_l \subset \mathcal{V}$, containing nodes with known class labels drawn from the set $\mathcal{Y} = \{y_1, y_2, \dots, y_c\}$, along with an unlabeled node set $\mathcal{V}_u \subset \mathcal{V} \setminus \mathcal{V}_l$, the objective of node classification is to accurately assign each node v to one of the predefined classes within $\mathcal{Y}$. This task serves as the foundation for numerous real-world applications, including social influence analysis, content recommendation, and predictive behavior modeling.

We assume that the server is an untrusted party, which has access to the node set $\mathcal{V}$ and the adjacency matrix $\mathbf{A}$, but cannot directly observe the sensitive node feature matrix $\mathbf{X}$. The feature data is decentralized and privately held by individual users. Protecting this information is essential for maintaining user confidentiality and mitigating the risk of data leakage.² Each user is associated with a personalized privacy level $\tau \in \{1, 2, \dots, h\}$, where a smaller value of τ indicates a stronger privacy preference, and thus requires injecting more noise. Users independently perturb their feature vectors based on their assigned τ using a LDP mechanism, enabling personalized privacy preservation. Importantly, although we focus on feature-level privacy, the graph structure, captured by the adjacency matrix $\mathbf{A}$, remains intact and is fully utilized during training. The server collects the perturbed features and performs graph learning using GNNs, which leverage the underlying topology for message passing and representation learning. Therefore, the graph component remains integral to the framework, and the interplay between noisy features and structural information is central to the model's design.

B. Graph Neural Networks

GNNs learn new node representations by combining initial node features and graph topology by aggregating node and neighbor information. These representations are then used for downstream machine learning tasks like node classification. A typical K -layer GNN consists of K graph convolution layers, where each layer aggregates information from a node's neighbors and updates the node's representation. After K aggregation iterations, the representation of a node captures the structural information in its K -hop neighborhood. The k -th layer of the GNN can be defined formally as follows:

$$\mathbf{h}_{\mathcal{N}(v)}^k = \text{AGG}_k(\{\mathbf{h}_u^{k-1}, \forall u \in \mathcal{N}(v)\}), \quad (1)$$

$$\mathbf{h}_v^k = \text{UPD}_k(\mathbf{h}_{\mathcal{N}(v)}^k, \Theta^k), \quad (2)$$

where $\mathcal{N}(v)$ is the set of neighbors of node v as well as could include v itself, and $\mathbf{h}_u^{k-1}$ is the embedding of an adjacent node u in the $k-1$ layer. $\text{AGG}_k(\cdot)$ denotes the aggregator function,

¹This paper uses 'node' and 'user' interchangeably, as in many applications, such as social networks, each node corresponds to a user.

²While this paper focuses on the privacy of node features, our framework is modular and can be seamlessly extended to incorporate edge privacy mechanisms such as those in [32], [33].

such as SUM, MEAN, or MAX. $\text{UPD}_k(\cdot)$ denotes a learnable update function, such as a neural network, parameterized by Θ^k . The above structured approach allows GNNs to effectively leverage both node features and relational information inherently present in the graph, thereby enhancing their performance on various tasks.

C. Local Differential Privacy

LDP, as a variant of DP, has been adopted as a powerful privacy-preserving method by companies such as Google [14], Apple [15] and Microsoft [16], and hundreds of millions of users' private information is being protected by this technology. In the LDP setting, the user perturbs the original data using a randomizer $\mathcal{R}$ on the user side and uploads the perturbed data to the server side. This ensures the privacy of user data, as only the data owner can access the original data [34]. The randomizer $\mathcal{R}$ is defined as follows:

Definition 1 (ϵ -LDP [35], [36]): A randomizer $\mathcal{R}$ satisfies ϵ -LDP, where $\epsilon > 0$, if and only if for any user's private data x and x' , and for all possible outputs $y \in \text{Range}(\mathcal{R})$:

$$\Pr[\mathcal{R}(x) = y] \leq e^\epsilon \cdot \Pr[\mathcal{R}(x') = y]. \quad (3)$$

The parameter ϵ , known as the *privacy budget*, plays a crucial role in LDP by regulating the balance between utility and privacy. A smaller (resp. larger) ϵ value provides stronger (resp. weaker) privacy guarantees but may result in lower (resp. higher) utility. The above definition implies that no matter what side knowledge the adversary has, they cannot, with high probability, infer the input value by observing the output. In addition, LDP satisfies several important properties [35] that are essential for subsequent proofs, as follows:

Proposition 1 (Sequential Composition [35]): Consider a sequence of computations $\mathcal{A}_1, \dots, \mathcal{A}_k$ applied to different components of the same local data record. If each $\mathcal{A}_i$ satisfies ϵ_i -LDP, then performing these computations sequentially ensures an overall privacy guarantee of $(\sum_i \epsilon_i)$ -LDP for that record.

Proposition 2 (Post-processing [35]): If an algorithm $\mathcal{A}(\cdot)$ satisfies ϵ -LDP, then any further processing of its output by another algorithm $\mathcal{B}(\cdot)$ (i.e., $\mathcal{B}(\mathcal{A}(\cdot))$) also maintains ϵ -LDP.

In real-world scenarios, users often exhibit varying levels of privacy sensitivity and expectations, resulting in diverse privacy requirements. The standard LDP framework assumes a uniform privacy budget for all users, which limits its applicability in personalized settings. To address this limitation, the notion of personalized LDP (PLDP) has been proposed [26], [27], [37], [38]. Under the PLDP setting, the server provides a set of discrete privacy levels, and each user can independently select a level $\tau \in \{1, \dots, h\}$ based on their individual privacy preferences. This selected level corresponds to a user-specific privacy budget ϵ^τ , allowing for flexible control over the noise injection process

while respecting user autonomy. Importantly, despite this personalization, PLDP retains the formal privacy guarantees and desirable properties of standard LDP [35].

D. Four Primary Node Feature LDP Mechanisms

1) **Laplace Mechanism:** The Laplace mechanism [13] is a classical LDP mechanism known for its simplicity. Assume each user possesses a one-dimensional value, denoted as x . Without loss of generality, we assume x is within the normalized value range of $[-1, 1]$. The subsequent three LDP mechanisms also adhere to this setting. Next, we define a randomization function that produces a perturbed value, denoted as $x' = x + \text{Lap}(2/\epsilon)$, where $\text{Lap}(\alpha)$ represents a random variable following a Laplace distribution with probability density function $f(x) = \frac{1}{2\alpha} \exp(-\frac{|x|}{\alpha})$. To extend to d -dimensional spaces, a straightforward approach is to apply the Laplacian mechanism independently to each dimension. Each dimension is allocated a privacy budget of ϵ/d . According to the sequential composition property [39] of the LDP mechanism, it adheres to ϵ -LDP.

2) **Gaussian Mechanism:** The Gaussian mechanism [21] is another well-established LDP mechanism recognized for its effectiveness and flexibility. Consider a scenario where each user has a one-dimensional value x . We define a randomization function that produces a perturbed value denoted as $x = x + N(0, \sigma^2)$, where $N(0, \sigma^2)$ is a Gaussian random variable with zero mean and variance σ^2 . In this context, the variance σ^2 is calibrated based on the privacy budget ϵ and the acceptable privacy leakage probability δ . Specifically, we denote σ^2 as $\sigma^2 = \frac{2 \ln(1.25/\delta)}{\epsilon^2}$, ensuring that (ϵ, δ) -LDP is satisfied, where $\delta > 0$ serves as a relaxed privacy parameter. For multidimensional data, the Gaussian mechanism is applied independently to each dimension, mirroring the approach used in the Laplace mechanism. In this case, the privacy budget for each dimension is allocated as ϵ/d . This independent application preserves the (ϵ, δ) -LDP properties due to the sequential combination nature of local differential privacy, thus ensuring robust privacy protection across all dimensions.

3) **1-Bit Mechanism:** In its one-dimensional form [16], for any original data $x \in [-1, 1]$, the distribution followed by the perturbed value $x' \in \{-1, 1\}$ is as follows:

$$\Pr[x' = c|x] = \begin{cases} \frac{1}{e^\epsilon + 1} + \frac{x+1}{2} \cdot \frac{e^\epsilon - 1}{e^\epsilon + 1}, & \text{if } c = 1 \\ \frac{1}{e^\epsilon + 1} - \frac{x+1}{2} \cdot \frac{e^\epsilon - 1}{e^\epsilon + 1}, & \text{if } c = -1 \end{cases}. \quad (4)$$

The extension to higher-dimensional spaces is implemented similarly to the first two mechanisms and is omitted here.

4) **Multi-Bit Mechanism:** The multi-bit mechanism [18], [22] extends the 1-bit mechanism [16] to higher-dimensional spaces through a sampling process. Specifically, it operates by uniformly sampling m -dimensions from d -dimensional node feature data without replacement. Each of the m selected dimensions is then perturbed using noise that satisfies ϵ/m -LDP. Due to the sequential composition property of LDP mechanisms [39], the overall process adheres to ϵ -LDP. After receiving all the perturbed data, the server transforms the reporting data $\hat{x}$ to its

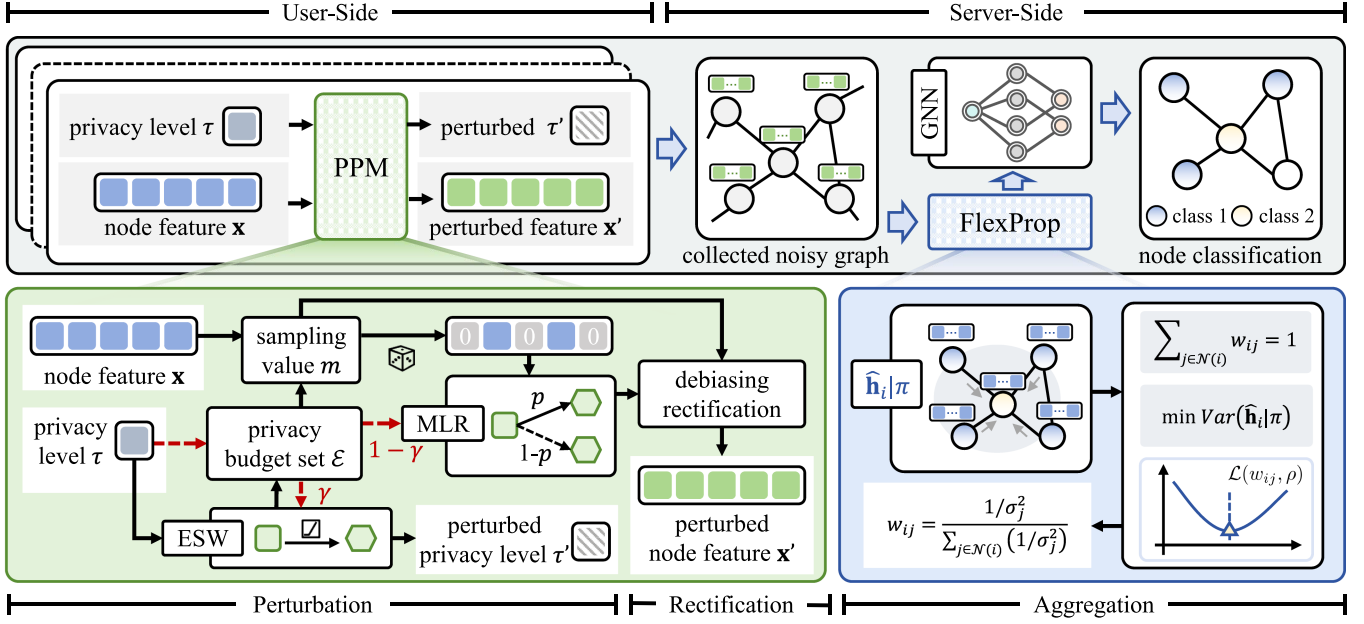


Fig. 2. Overview of our proposed PPGNN. Users run the Personalized Perturbation Mechanism (PPM) to their private data (node features $\mathbf{x}$ and privacy levels τ) and send the output to the server. The server calibrates the collected noisy data using the FlexProp algorithm and performs GNN training for the node classification task. The red dashed path illustrates the allocation scheme for privacy budgets.

unbiased estimate $x' = \frac{d}{m} \cdot \frac{e^{\epsilon/m} + 1}{e^{\epsilon/m} - 1} \cdot \hat{x}$. The variance of x' is as follows: $\text{Var}[x'] = \frac{d}{m} \cdot \left(\frac{e^{\epsilon/m} + 1}{e^{\epsilon/m} - 1} \right)^2 - x^2$.

III. METHODOLOGY

In this section, we introduce PPGNN, a locally differentially private graph learning model designed to effectively meet users' personalized privacy requirements while significantly enhancing the utility of private graph learning. Next, we first introduce the overall framework of PPGNN in Section III-A, followed by a detailed discussion of the two key steps of PPGNN in Sections III-B and III-C, respectively. Finally, we conduct complexity analysis in Section III-D and privacy analysis in Section III-E.

A. Overview

In the scenario of locally private graph learning, the node features located on the user side are transmitted to the server after undergoing noise injection. The server performs private graph learning based on the collected node features. The overall framework of PPGNN is illustrated in Fig. 2. PPGNN addresses users' personalized privacy requirements effectively and enhances the utility of privacy graph learning through adaptive personalized aggregation. Specifically, in PPGNN, each user's local data consists of node features and privacy levels, which are protected using a *Personalized Perturbation Mechanism (PPM)* that dynamically adjusts perturbation levels based on individual privacy preferences. On the server side, the *FlexProp* algorithm is applied to correct the noisy node features, mitigating the biases introduced by the perturbation process. Once corrected, the server employs a GNN to train on the refined data, enabling

Algorithm 1: Local Randomizer.

Input: Numeric data $x \in [-1, 1]$, privacy budget ϵ .

Output: Randomized response $x' \in \left\{ -\frac{e^\epsilon + 1}{e^\epsilon - 1}, \frac{e^\epsilon + 1}{e^\epsilon - 1} \right\}$.

- 1: Sample a Bernoulli variable u such that $\Pr[u = 1] = \frac{e^\epsilon - 1}{2e^\epsilon + 2} \cdot x + \frac{1}{2}$.
- 2: **if** $u = 1$ **then**
- 3: $x' = (e^\epsilon + 1)/(e^\epsilon - 1)$.
- 4: **else**
- 5: $x' = -(e^\epsilon + 1)/(e^\epsilon - 1)$.
- 6: **end if**
- 7: **return** x'

downstream tasks such as node classification. PPGNN integrates personalized privacy protection with adaptive feature correction, achieving a balance between robust privacy guarantees and high utility, thus optimizing the performance of privacy-preserving graph learning. In summary, PPGNN consists of the following two parts:

1) *Perturbing Node Features With PPM*: Although several LDP mechanisms applicable to node features have been proposed (as outlined in Section II-D), these mechanisms fail to address the dual privacy requirements inherent in personalized privacy graph learning scenarios. Additionally, there remains significant potential to improve the utility of these mechanisms by reducing noise perturbation variance to enhance data accuracy. Balancing these challenges is non-trivial, as it requires simultaneously minimizing noise and preserving dual privacy. To tackle this issue, we introduce a novel LDP perturbation algorithm, the *Personalized Perturbation Mechanism (PPM)*, which

incorporates two interdependent modules: a multi-dimensional local randomizer and an extended square-wave mechanism. These modules work synergistically to effectively reduce noise variance while addressing dual privacy concerns. In the PPGNN framework, each user perturbs their local data using PPM before transmitting the processed data to the server for further private graph learning.

2) *Calibrating Noise Features With FlexProp*: Directly utilizing the noisy data generated by the LDP mechanism for graph learning often leads to poor utility, necessitating further calibration to reduce noise. To address this, we employ multi-hop aggregation, which is theoretically effective in minimizing estimation errors. However, traditional multi-hop aggregation is suboptimal in personalized privacy-preserving scenarios, as it fails to leverage the effective information of each node fully. To overcome this limitation, we propose the FlexProp algorithm, which efficiently minimizes estimation errors through adaptive weighted aggregation, tailoring the process to individual nodes' privacy requirements and data features.

B. Perturbing Node Features With PPM

To achieve personalized privacy preservation for node features, we introduce a novel LDP algorithm called the *Personalized Perturbation Mechanism (PPM)*, as outlined in Algorithm 3. The architectural design of PPM is illustrated in Fig. 2. In this framework, we strategically divide the privacy budget into h discrete levels, denoted as $\mathcal{E} = \cup_{\tau=1}^h \{\epsilon^\tau\}$. Here, $\Delta_\epsilon = \epsilon^{\tau+1} - \epsilon^\tau > 0$ indicates that each subsequent level represents a progressively greater privacy budget, while ϵ^τ denotes the specific budget at privacy level τ . Notably, a smaller value of τ corresponds to a reduced privacy budget, leading to the injection of a greater amount of noise. As depicted in Fig. 2, the PPM consists of two critical components: the *Multi-dimensional Local Randomizer (MLR)* and the *Extended Square Wave (ESW)* mechanism. The allocation of the privacy budget between these two components is governed by the parameter γ . Specifically, the ESW utilizes $\gamma\epsilon^\tau$ of the privacy budget, while the MLR is allocated $(1 - \gamma)\epsilon^\tau$. This careful distribution ensures that each component effectively contributes to the overall privacy protection mechanism, balancing the trade-off between noise injection and data utility. Next, we explore the details of MLR and ESW respectively.

1) *Multi-Dimensional Local Randomizer*: To protect multi-dimensional node features, we introduce the MLR algorithm, as presented in Algorithm 2. This algorithm extends the concept of the *local randomizer* (Algorithm 1) [40], originally designed for unidimensional spaces, to accommodate multidimensional contexts. The MLR algorithm³ effectively allocates the total privacy budget across m attributes, rather than the original d , which significantly reduces the noise variance associated with the perturbation process. As a trade-off, the additional estimation error introduced by sampling m attributes from d can be

³The MLR algorithm is not limited to the local randomizer in [40]; it serves as a general wrapper that can integrate any one-dimensional fixed-budget perturbation mechanism. This modularity allows MLR to be adapted to diverse privacy mechanisms (e.g., Laplace).

Algorithm 2: Multi-Dimensional Local Randomizer.

Input: Feature vector $\mathbf{x} \in [-1, 1]^d$, privacy budget ϵ

Output: $\mathbf{x}' \in \left\{ -\frac{d}{m} \cdot \frac{e^{\epsilon/m} + 1}{e^{\epsilon/m} - 1}, 0, \frac{d}{m} \cdot \frac{e^{\epsilon/m} + 1}{e^{\epsilon/m} - 1} \right\}^d$

- 1: Let $\mathbf{x}' = \langle 0, 0, \dots, 0 \rangle$.
 - 2: Let $m = \max(1, \min(d, \lfloor \epsilon/2.2 \rfloor))$.
 - 3: Sample m values uniformly without replacement from $\{1, 2, \dots, d\}$.
 - 4: **for** each sampled value j **do**
 - 5: Feed $\mathbf{x}_j$ and ϵ/m to the Algorithm 1 to obtain t_j .
 - 6: $\mathbf{x}'_j = \frac{d}{m} \cdot t_j$.
 - 7: **end for**
 - 8: **return** $\mathbf{x}'$
-

managed by judiciously selecting an appropriate m value. This balancing act is formally addressed in Theorem 1. Furthermore, Theorem 2 establishes that the MLR algorithm satisfies ϵ -LDP and proves that $\mathbb{E}[\mathbf{x}'] = \mathbf{x}$.

Theorem 1: To minimize the variance of the perturbation mechanism (as given by (5)), the optimal sampling parameter $m = \max(1, \min(d, \lfloor \epsilon/2.2 \rfloor))$.

$$\text{Var}[\mathbf{x}'_i] = \frac{d}{m} \cdot \left(\frac{e^{\epsilon/m} + 1}{e^{\epsilon/m} - 1} \right)^2 - \mathbf{x}_i^2. \quad (5)$$

Proof. Given that variance is a critical factor that affects the accuracy of the estimation of the LDP mechanisms: the smaller the variance, the more accurate the estimation, the theorem selects the optimal value of m by minimizing the variance of the LDP mechanism. See Appendix A, available online for more details. $\square$

Theorem 2: Algorithm 2 (MLR) satisfies ϵ -LDP for each node. In addition, for any $i \in \{1, 2, \dots, d\}$, $\mathbb{E}[\mathbf{x}'_i] = \mathbf{x}_i$.

Proof: We denote the MLR algorithm as $\mathcal{R}^1$. According to Definition 1, our objective is to demonstrate that for any two input features $\mathbf{x}_1$ and $\mathbf{x}_2$ satisfying the following inequality: $\Pr[\mathcal{R}^1(\mathbf{x}_1) = \mathbf{x}'] \leq e^\epsilon \cdot \Pr[\mathcal{R}^1(\mathbf{x}_2) = \mathbf{x}']$, where $\mathbf{x}'$ represents a perturbed output. Furthermore, ensuring that the data are unbiased (that is, $\mathbb{E}[\mathbf{x}'_i] = \mathbf{x}_i$) guarantees the statistical utility of the data. We have also provided a proof of this property. For further details, please refer to the Appendix B, available online. $\square$

Note that the user's privacy level τ is also sensitive and utilized in the subsequent FlexProp algorithm. Directly applying the MLR algorithm to protect node features may potentially reveal τ , as τ determines ϵ^τ , which in turn affects the final output. To address this issue, we propose the Extended Square Wave (ESW) mechanism. Additionally, considering that m could potentially leak the user's privacy level, instead of directly using τ , we use a randomized version τ' through the ESW, that is: $m' = \max(1, \min(d, \lfloor \epsilon^{\tau'}/2.2 \rfloor))$.

2) *Extended Square Wave Mechanism*: To protect the users' privacy levels, the ESW extends from the square wave mechanism [31] and can be applied to the protection of discrete domain data after extension. In ESW, the input domain is defined as $\mathcal{D} = \{1, 2, \dots, f\}$, where f is the total number of privacy levels, and the input value $g \in \mathcal{D}$ represents a user's true privacy level index. The output domain is extended to $\mathcal{D}' = \{1, 2, \dots, f + 2b\}$, where $g' \in \mathcal{D}'$ is the perturbed output value. The parameter b controls the width of the high-probability window around g and is defined as: $b = \lfloor \frac{e^\epsilon - e^\epsilon + 1}{2e^\epsilon(e^\epsilon - 1 - e)} \cdot f \rfloor$. This extension allows the mechanism to assign higher output probability near the true value to preserve utility, while still satisfying ϵ -LDP through randomization across a larger space. The perturbation process is as follows:

$$\Pr[\text{ESW}(g) = g'] = \begin{cases} p, & \text{if } |g - g'| \leq b \\ q, & \text{otherwise} \end{cases}, \quad (6)$$

where $p = e^\epsilon / ((2b + 1)e^\epsilon + f - 1)$, $q = 1 / ((2b + 1)e^\epsilon + f - 1)$. The values of p and q ensure ϵ -local differential privacy over the extended output domain $\mathcal{D}'$, while the use of the high-probability window improves the fidelity of perturbed outputs.

C. Calibrating Noise Features With FlexProp

In this section, we first conduct an error analysis to identify the key factors influencing the utility enhancement of perturbed node features. Building on these insights, we introduce FlexProp, a weighted aggregation algorithm specifically designed for personalized scenarios, which efficiently calibrates the perturbed node features to maximize utility while respecting individual privacy requirements.

1) *Error Analysis*: The server proceeds with GNN training after receiving all the perturbed node features. During the first layer of GNN, the embedding for any given node $v \in \mathcal{V}$ is produced through the following process:

$$\hat{\mathbf{h}}_{\mathcal{N}(v)} = \text{AGG}(\{\mathbf{x}'_u, \forall u \in \mathcal{N}(v)\}), \quad (7)$$

where $\hat{\mathbf{h}}_{\mathcal{N}(v)}$ is the estimation of the $\text{AGG}(\cdot)$ function that aggregates the feature vectors $\mathbf{x}'_u$ of all perturbed nodes $u \in \mathcal{N}(v)$. The aggregation process provides an unbiased estimate when $\mathbb{E}[\mathbf{x}'_u] = \mathbf{x}_u$ (Theorem 2) and the AGG function operates linearly, as established in Theorem 3.

Theorem 3: The linear aggregation function described by (7) guarantees unbiased estimation. Specifically, for any node $v \in \mathcal{V}$, it holds that: $\mathbb{E}[\hat{\mathbf{h}}_{\mathcal{N}(v)}] = \mathbf{h}_{\mathcal{N}(v)}$.

Proof: See Appendix C, available online for the proof. $\square$

The estimation error Δ between the estimated node embedding $\hat{\mathbf{h}}_{\mathcal{N}(v)}$ and the true node embedding $\mathbf{h}_{\mathcal{N}(v)}$ for any node v is defined as the maximum absolute difference across each dimension i in the embedding, expressed as:

$$\Delta = \max_{i \in \{1, 2, \dots, d\}} |(\hat{\mathbf{h}}_{\mathcal{N}(v)})_i - (\mathbf{h}_{\mathcal{N}(v)})_i|. \quad (8)$$

where $(\hat{\mathbf{h}}_{\mathcal{N}(v)})_i$ and $(\mathbf{h}_{\mathcal{N}(v)})_i$ represent the i -th dimension of the estimated and true node embeddings, respectively. This error quantifies the deviation introduced by the perturbation and

subsequent aggregation process. The detailed analysis of Δ is provided in Theorem 4.

Theorem 4: Let $\text{AGG}(\cdot)$ be the MEAN aggregator and $\delta > 0$, with at least $1 - \delta$ probability, the estimation error Δ for any given node v is: $\Delta = \mathcal{O}(\frac{\sqrt{d \log(d/\delta)}}{(1-\gamma)\epsilon \sqrt{|\mathcal{N}(v)|}}) + \mathcal{O}(\frac{1}{\gamma\epsilon})$.

Proof: See Appendix D, available online for the proof. $\square$

The above theorem indicates that, given a fixed ϵ , selecting an appropriate γ and expanding the size of $\mathcal{N}(v)$ are both beneficial in reducing the estimation error. The trade-offs of γ are discussed in Section IV-F3.

2) *FlexProp Algorithm*: To further minimize the error, we introduce a denoising aggregation algorithm called FlexProp, which calibrates noise by aggregating larger neighborhoods. Previous works [18], [28], [29], [30] have shown that considering higher-order neighborhoods aids in learning superior node representations. As illustrated in Algorithm 4, for a node v , FlexProp aggregates features of nodes up to the farthest K steps away from v by invoking the AGG function K consecutive times, without any non-linear transformation in between, which facilitates noise calibration. Additionally, FlexProp examines the influence of aggregation policy π on enhancing aggregation accuracy. In a personalized privacy-preserving context, varying aggregation policies can substantially impact accuracy due to differing noise scales among neighboring nodes. We define the estimated feature of node i post-aggregation under policy π as: $\hat{\mathbf{h}}_i(\pi) = \text{AGG}(\{\mathbf{x}'_j, \forall j \in \mathcal{N}(i)\})$. Assuming w_{ij} represents the weight of node j when aggregated to target node i , and considering a weighted aggregation function, we have:

$$\hat{\mathbf{h}}_i(\pi) = \frac{1}{\sum_{j \in \mathcal{N}(i)} w_{ij}} \sum_{j \in \mathcal{N}(i)} w_{ij} \mathbf{x}'_j. \quad (9)$$

Furthermore, assuming the feature vector $\mathbf{x}'_i$ of each node i decomposes into the true signal $\mathbf{s}_i$ and noise $\mathbf{n}_i$, i.e., $\mathbf{x}'_i = \mathbf{s}_i + \mathbf{n}_i$, where $\mathbf{n}_i$ is a random variable with a mean of 0 and a variance of σ_i^2 , we have: $\hat{\mathbf{h}}_i(\pi) = \frac{1}{\sum_{j \in \mathcal{N}(i)} w_{ij}} \sum_{j \in \mathcal{N}(i)} w_{ij} (\mathbf{s}_j + \mathbf{n}_j)$.

Next, we analyze the variance of the noise after aggregation, denoted as $\text{Var}(\hat{\mathbf{h}}_i(\pi))$, to better assess the denoising effectiveness and determine the optimal aggregation policy. The variance is calculated as follows:

$$\text{Var}(\hat{\mathbf{h}}_i(\pi)) = \frac{1}{\left(\sum_{j \in \mathcal{N}(v)} w_{ij}\right)^2} \sum_{j \in \mathcal{N}(v)} w_{ij}^2 \sigma_j^2. \quad (10)$$

To achieve optimal noise removal, the variance of the aggregated noise should be minimized. We aim to select a set of weights w_{ij} that satisfy the constraint $\sum_{j \in \mathcal{N}(i)} w_{ij} = 1$ while minimizing this variance. To find the optimal solution, we employ the *Lagrange multiplier* method [41]. The Lagrange function is defined as:

$$\mathcal{L}(w_{ij}, \rho) = \sum_{j \in \mathcal{N}(i)} w_{ij}^2 \sigma_j^2 + \rho \left(\sum_{j \in \mathcal{N}(i)} w_{ij} - 1 \right). \quad (11)$$

Algorithm 3: Personalized Perturbation Mechanism.

Input: User's feature vector $\mathbf{x} \in [-1, 1]^d$, privacy level $\tau \in \{1, 2, \dots, h\}$, privacy budget set $\mathcal{E} = \cup_{\tau=1}^h \{\epsilon^\tau\}$, allocation parameter γ

Output: Perturbed node feature $\mathbf{x}'$ and τ'

- 1: $\epsilon^\tau \leftarrow \mathcal{E}(\tau)$. /* total privacy budget */
- 2: $\epsilon_1 \leftarrow \gamma \epsilon^\tau$. /* privacy budget of the privacy level */
- 3: $\epsilon_2 \leftarrow (1 - \gamma) \epsilon^\tau$. /* privacy budget of the node feature vector */
- 4: $\tau' \leftarrow \text{ESW}(\tau, \epsilon_1)$ and $\mathbf{x}' \leftarrow \text{MLR}(\mathbf{x}, \epsilon_2)$.
- 5: **return** $\mathbf{x}'$ and τ'

The optimal weights are obtained by solving the above equation as $w_{ij} = \frac{1/\sigma_j^2}{\sum_{j \in \mathcal{N}(i)} (1/\sigma_j^2)}$. For more details, please refer to the Appendix E, available online.

In practice, the optimal weights under the given policy π are not always feasible. To address this, we approximate the optimal weights using the neighbor aggregation function Π and incorporate it into Algorithm 4.⁴ The function Π can be represented in matrix form as follows,

$$\Pi_{i,j} = \begin{cases} 0, & \text{if } \mathbf{A}_{i,j} = 0 \\ \alpha_{i,j}, & \text{if } \mathbf{A}_{i,j} = 1 \end{cases}, \quad (12)$$

where $\mathbf{A}$ is the adjacency matrix and $\alpha_{i,j} = \tau'_j / \sum_{j \in \mathcal{N}(i)} \tau'_j$.

D. Complexity Analysis

The time complexity of PPGNN is determined by its two key components: the PPM and the FlexProp aggregation algorithm. PPM perturbs the privacy level and feature vector of each node locally, requiring $\mathcal{O}(|\mathcal{V}| \cdot d)$, where $|\mathcal{V}|$ is the number of nodes and d is the feature dimension. FlexProp performs K layer message aggregation, involving $\mathcal{O}(K \cdot E \cdot d)$, where E is the number of edges and K is the number of layers. Combining these, the overall time complexity is $\mathcal{O}(|\mathcal{V}| \cdot d + K \cdot E \cdot d)$, scaling linearly with graph size and feature dimensionality. This linear scalability ensures computational efficiency in practice and makes PPGNN suitable for real-world applications involving large, high-dimensional graphs.

E. Privacy Analysis

Recalling the definition of LDP (Definition 1), the PPGNN satisfies ϵ^τ -LDP as established by Theorem 5, which relies on the composition property (Proposition 1) and robustness to post-processing (Proposition 2) in differential privacy [35]. Furthermore, any subsequent predictions made by the PPGNN are governed by the post-processing theorem, since the LDP mechanism is applied only once to the private data.

⁴Unlike the KProp Layer in [18], which performs multi-hop aggregation under a global privacy budget, our FlexProp is tailored for personalized LDP scenarios, where users have heterogeneous noise levels. FlexProp introduces a privacy-aware weight matrix Π to calibrate the aggregation process while preserving privacy, which is not considered in KProp.

Algorithm 4: FlexProp.

Input: Graph $\mathcal{G} = (\mathcal{V}, \mathbf{A})$, privacy level $\tau_v, \forall v \in \mathcal{V}$, input features $\mathbf{x}_v$, aggregation function AGG, propagation step $K \geq 0$

Output: Final embedding vector $\mathbf{h}_v, \forall v \in \mathcal{V}$

- 1: Compute propagation weights Π using (12)
- 2: **for each** node $v \in \mathcal{V}$ **do**
- 3: Initialize hidden representation: $\mathbf{h}_v^0 = \mathbf{x}_v$
- 4: **end for**
- 5: **for** $k = 1$ **to** K **do**
- 6: **for each** node $v \in \mathcal{V}$ **do**
- 7: $\mathbf{h}_v^k = \text{AGG}(\{\Pi_{v,u} \cdot \mathbf{h}_u^{k-1} \mid u \in \mathcal{N}(v)\})$
- 8: **end for**
- 9: **end for**
- 10: **return** $\mathbf{h}_v^K, \forall v \in \mathcal{V}$

TABLE II
STATISTICS OF GRAPH DATASETS

DATASET	#CLASSES	#NODES	#EDGES	#FEATURES
CORA	7	2,708	5,278	1,433
CITESEER	6	3,327	4,552	3,703
PUBMED	3	19,717	44,324	500
LASTFM	18	7,624	27,806	7,842
FACEBOOK	4	22,470	170,912	4,714
WIKIPEDIA	2	11,631	170,845	13,183

Theorem 5: PPGNN satisfies ϵ^τ -LDP for each node.

Proof: See Appendix F, available online for the proof. $\square$

IV. EXPERIMENTS

A. Experimental Settings

1) *Datasets:* Our experiments use six publicly real-world datasets: Cora [42], CiteSeer [42], Pubmed [42], LastFM [43], Facebook [44], and Wikipedia [44]. The statistical details of these datasets are presented in Table II.

- *Cora* [42]: A citation network in which each node corresponds to a scientific paper and edges signify citation links between papers. Node features are derived from paper content, and each node is labeled with a research topic.
- *CiteSeer* [42]: Also a citation network where nodes represent academic papers and edges denote citation relationships. Each node has a content-based feature vector and belongs to a research category.
- *Pubmed* [42]: A large-scale citation network in which nodes are biomedical papers and edges represent citations. Node features are TF/IDF weighted term vectors, and the task is multi-class classification among diabetes-related topics.
- *LastFM* [43]: A social network where nodes correspond to LastFM music platform users and edges reflect friendship connections. The task is to predict each user's home country based on the artists they have liked.
- *Facebook* [44]: A social network in which each node is an official Facebook page and edges represent mutual

“likes” between pages. Node features are derived from page descriptions, and the task is page-type classification.

- **Wikipedia [44]:** A co-occurrence network constructed from Wikipedia pages, where nodes represent articles and edges connect pages that share common links. The task is binary classification of page category.

2) **Baselines:** To demonstrate the effectiveness of PPGNN, we compare it with the following baselines: *NonPriv*, *BASE*, and *LPGNN* [18].⁵ The *NonPriv* employs clean node features for graph learning. In contrast, *BASE* uses features perturbed by the LDP mechanism directly in graph learning, bypassing a separate noise calibration step. *LPGNN*, recognized as the current state-of-the-art in locally private graph learning, perturbs node features using a multi-bit mechanism and calibrates noise using the KProp algorithm. Additionally, to evaluate the performance of PPM, we compare it with the 1-bit mechanism (*1B*) [16], Laplace mechanism (*LP*) [34], Gaussian mechanism (*GM*) [21], and multi-bit mechanism (*MB*) [18].

3) **Experiment Setup:** For each dataset, we randomly divide 50% of the data samples as the training set, 25% as the validation set and 25% as the test set. We select three GNN architectures as backbone models: **GCN** [4], which aggregates neighborhood information via a symmetric normalized adjacency matrix; **GraphSAGE** [2], which samples a fixed-size neighborhood and aggregates features using learnable aggregator functions; and **GAT** [45], which introduces an attention mechanism to assign learned importance weights to different neighbors during aggregation. All GNN models have two graph convolutional layers, each with a hidden dimension of size 16 and a SeLU activation function followed by dropout. We implement the GNN models in PyTorch using PyTorch-Geometric (PyG).⁶ All experiments are carried out on a machine running Ubuntu 20.04 LTS, equipped with two Intel Xeon Gold 6348 CPUs, 100 GB RAM, and an NVIDIA A800 80 GB GPU. We set $\mathcal{E} = \{\epsilon, 2\epsilon, \dots, 2^h\epsilon\}$, where $\epsilon \in \{0.01, 1, 2, 3\}$. If not specified otherwise, h is set to 5. To obtain the best hyperparameters, we use grid search for selection: both learning rate and weight decay are chosen from $\{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 0\}$, the dropout rate is chosen from $\{0.75, 0.5, 0.25, 0\}$, and the allocation parameter δ is chosen from $\{0.1, 0.3, 0.5, 0.7, 0.9\}$. The optimal value of the FlexProp’s step parameter K for different $\epsilon \in \{0.01, 1, 2, 3\}$ is selected from the set $\{0, 2, 4, 8, 16\}$. We used the Adam optimizer [46] for all models. We utilize a random function to generate users’ privacy levels.

4) **Evaluation Metrics:** We use node classification accuracy as a metric to assess the performance of PPGNN with different parameters. All models are trained for 500 epochs, and the best model is selected for testing based on validation loss. We measure the accuracy of the test set over 10 consecutive runs and report the average and 95% confidence intervals calculated by bootstrapping over 1000 samples.

⁵Considering that the node feature protection method in [19] aligns with LPGNN [18], and the research context of [22] focuses on subgraph-level protection, a direct comparison is not necessary. Therefore, these works are not included as baselines in this study.

⁶<https://www.pyg.org>

B. Main Experimental Results

Fig. 3 demonstrates the accuracy of PPGNN with various baselines for all cases. In this experiment, the BASE method employs the MB [18] to perturb node features. As depicted in Fig. 3, PPGNN consistently outperforms both BASE and LPGNN, significantly narrowing the gap with NonPriv. Notably, in some cases, the accuracy of PPGNN closely approaches that of NonPriv. Specifically, as shown in Fig. 3(a), when $\epsilon = 0.01$, LPGNN and PPGNN exhibit accuracy 24.5% and 26.8% higher than that of BASE, respectively. This not only underscores the effectiveness of the noise calibration strategies employed by LPGNN and PPGNN but also emphasizes that relying solely on the LDP mechanism, without adequate noise calibration, leads to diminished utility. Furthermore, as evidenced in Fig. 3(e) and (k), PPGNN’s accuracy aligns closely with that of NonPriv. Collectively, these observations validate the robust efficacy of PPGNN.

To further understand the impact of different GNN architectures under personalized LDP settings, we compare the performance of PPGNN when instantiated with GCN [4], GraphSAGE [2], and GAT [45] backbones. As shown in the left panel of Fig. 4, all three models benefit from the proposed framework, but their sensitivity to noise varies. Notably, GAT tends to underperform GCN and GraphSAGE when the ϵ is small (e.g., 0.01). We attribute this to GAT’s attention mechanism, which dynamically learns neighbor importance scores based on feature similarity. When input features are heavily perturbed, this attention computation becomes unstable, resulting in greater performance degradation. In contrast, GCN and GraphSAGE adopt more stable aggregation strategies, making them less sensitive to noise introduced by local perturbations. However, as ϵ increases (i.e., less noise), this gap gradually narrows, and the three models converge in performance.

C. Robustness Under Extreme Privacy Distributions

To further evaluate the adaptability of PPGNN in realistic settings, we simulate three extreme user privacy level distributions on the Cora dataset with GCN: ① *Strict Privacy*: all users adopt the lowest privacy budget ϵ ; ② *Relaxed Privacy*: all users adopt the highest budget $2^h\epsilon$; ③ *Bimodal*: half of the users adopt ϵ , the other half $2^h\epsilon$. As shown in the right panel of Fig. 5, PPGNN consistently outperforms BASE in all cases. Under strict privacy (Case 1), PPGNN and LPGNN achieve similar performance, while both clearly surpass BASE. When privacy becomes more relaxed (Cases 2 and 3), PPGNN benefits more from its personalized design and achieves the best utility. These results confirm that PPGNN remains effective and robust under diverse privacy-level distributions.

D. Ablation Study

1) *Analyzing the Performance of PPM:* In Table III, we compare the performance of our Personalized Perturbation Mechanism (PPM) with the 1-bit mechanism (1B), the Laplace mechanism (LP), the Gaussian mechanism (GM), and the Multi-bit mechanism (MB). 1B [16], GM [21], and LP [34] are widely

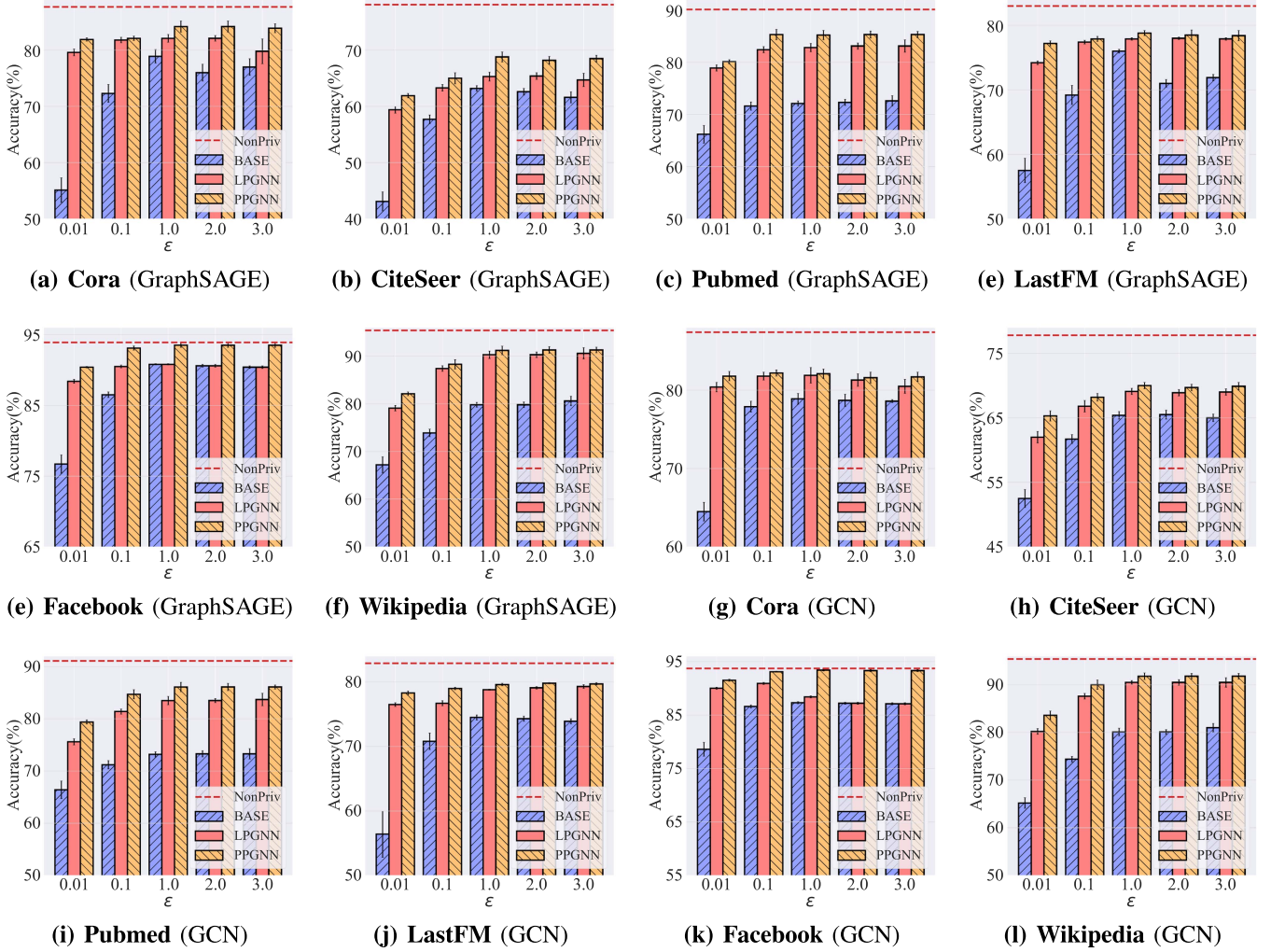


Fig. 3. Performance of PPGNN and other baselines. X-axis represents ϵ and y-axis represents test accuracy (%).

TABLE III
ACCURACY OF DIFFERENT FEATURE LDP MECHANISMS

ϵ	MECH.	CORA	CITESEER	PUBMED	LASTFM	FACEBOOK	WIKIPEDIA
$\epsilon = 0.01$	1B	25.9 $\pm$ 2.8	20.6 $\pm$ 1.2	52.5 $\pm$ 1.2	12.2 $\pm$ 3.3	30.3 $\pm$ 1.2	42.2 $\pm$ 2.5
	LP	23.3 $\pm$ 3.2	20.6 $\pm$ 1.3	58.1 $\pm$ 0.9	9.0 $\pm$ 1.8	31.5 $\pm$ 1.3	43.5 $\pm$ 2.1
	GM	31.3 $\pm$ 2.6	25.4 $\pm$ 1.8	59.2 $\pm$ 1.5	28.2 $\pm$ 2.1	44.7 $\pm$ 1.3	51.4 $\pm$ 2.5
	MB	55.1 $\pm$ 2.2	43.1 $\pm$ 1.7	66.2 $\pm$ 0.7	62.5 $\pm$ 1.9	76.7 $\pm$ 1.3	63.6 $\pm$ 2.3
	PPM	62.0 $\pm$ 2.1	47.3 $\pm$ 1.8	68.6 $\pm$ 1.1	66.4 $\pm$ 1.6	79.5 $\pm$ 1.4	67.2 $\pm$ 2.6
$\epsilon = 0.1$	1B	30.2 $\pm$ 1.2	22.5 $\pm$ 1.0	57.5 $\pm$ 2.3	18.8 $\pm$ 2.8	34.5 $\pm$ 0.9	47.1 $\pm$ 2.2
	LP	32.5 $\pm$ 0.9	21.0 $\pm$ 1.5	56.1 $\pm$ 2.1	18.3 $\pm$ 1.6	35.1 $\pm$ 1.3	49.2 $\pm$ 2.2
	GM	37.7 $\pm$ 1.2	27.8 $\pm$ 0.9	62.2 $\pm$ 1.8	41.2 $\pm$ 1.1	61.4 $\pm$ 0.8	62.6 $\pm$ 2.1
	MB	72.3 $\pm$ 1.6	57.7 $\pm$ 0.8	71.6 $\pm$ 1.9	74.2 $\pm$ 1.5	86.5 $\pm$ 0.4	72.3 $\pm$ 1.8
	PPM	76.1 $\pm$ 0.8	61.0 $\pm$ 0.8	75.3 $\pm$ 2.2	78.1 $\pm$ 0.9	88.7 $\pm$ 0.3	73.9 $\pm$ 2.5
$\epsilon = 1.0$	1B	42.8 $\pm$ 1.6	27.1 $\pm$ 1.4	62.4 $\pm$ 3.3	31.0 $\pm$ 3.6	63.2 $\pm$ 1.5	55.6 $\pm$ 1.7
	LP	39.7 $\pm$ 3.0	25.8 $\pm$ 2.1	62.7 $\pm$ 2.9	29.4 $\pm$ 3.6	58.7 $\pm$ 2.2	58.4 $\pm$ 2.2
	GM	46.5 $\pm$ 2.1	31.2 $\pm$ 1.8	65.1 $\pm$ 1.4	49.1 $\pm$ 2.5	71.2 $\pm$ 1.4	70.2 $\pm$ 1.9
	MB	78.9 $\pm$ 1.2	63.2 $\pm$ 0.5	72.1 $\pm$ 2.4	81.0 $\pm$ 0.3	90.7 $\pm$ 0.1	74.1 $\pm$ 1.8
	PPM	79.1 $\pm$ 0.5	63.3 $\pm$ 0.5	75.4 $\pm$ 2.1	81.2 $\pm$ 0.6	90.9 $\pm$ 0.2	79.8 $\pm$ 1.9

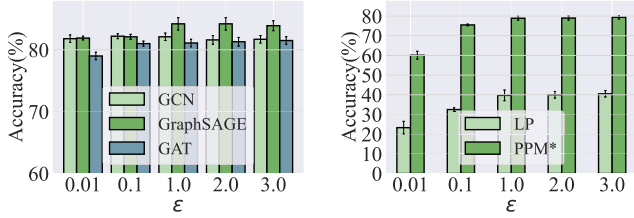


Fig. 4. **Left:** Comparison of the performance of PPGNN under different GNN models. **Right:** Performance of PPM* (LP integrated) vs. vanilla LP under different ϵ values, demonstrating utility gains from integrating LP into MLR.

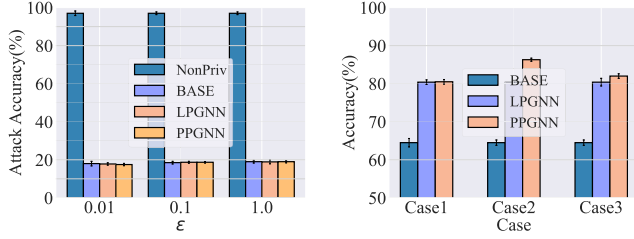


Fig. 5. **Left:** Attack accuracy of attribute inference under different privacy budgets (ϵ) on the Cora dataset. LDP-based methods (BASE, LPGNN, and PPGNN) effectively mitigate privacy leakage compared to NonPriv, while PPGNN preserves strong utility without compromising privacy. **Right:** Accuracy of PPGNN under three extreme privacy distributions on the Cora dataset using GCN. PPGNN demonstrates strong adaptability across different user privacy configurations, validating its personalized design.

used LDP mechanisms for locally perturbing single-value and multidimensional data. In this experiment, 1B, GM, and LP perturb each bit of the node features. MB is the LDP mechanism employed in LPGNN and currently stands as the most effective LDP algorithm for node features. According to the results in Table III, the accuracy of PPM surpasses that of other mechanisms across almost all cases, especially under smaller ϵ . Specifically, when $\epsilon = 0.01$ and the dataset is Cora, PPM achieves an accuracy that is approximately 7%, higher accuracy on average compared to the second-best mechanism. This underscores the efficacy of PPM. To further demonstrate the effectiveness and generality of our MLR-based design, we conduct a more detailed analysis in Fig. 4 Right, where we compare PPM* (i.e., LP [34] integrated into the MLR) against the vanilla LP across different ϵ . As shown in the figure, PPM* consistently outperforms LP, especially under tighter budgets. This verifies the utility gains and broad applicability of MLR when combined with various perturbation strategies.

2) *Analyzing the Performance of FlexProp:* In Figs. 8 and 9, we contrast our FlexProp with another noise calibration algorithm for node features, KProp, which is the noise calibration algorithm used in LPGNN [18]. Based on the results from Figs. 8 and 9, the accuracy of FlexProp surpasses that of KProp across all cases, underscoring the effectiveness of FlexProp. Additionally, we observe that FlexProp exhibits a more significant improvement with a larger privacy budget ϵ . This is attributed to the reduced noise injection due to the higher ϵ , allowing FlexProp to demonstrate its advantages more prominently, particularly in

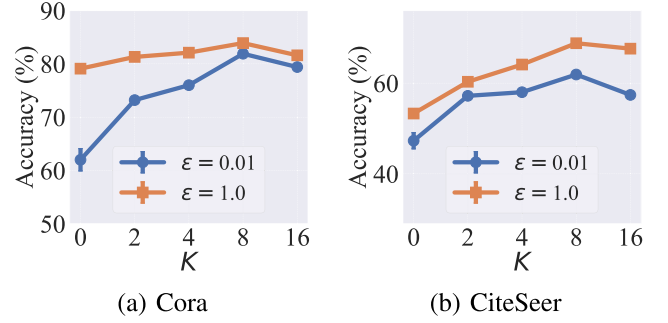


Fig. 6. Accuracy of PPGNN across varied parameter K .

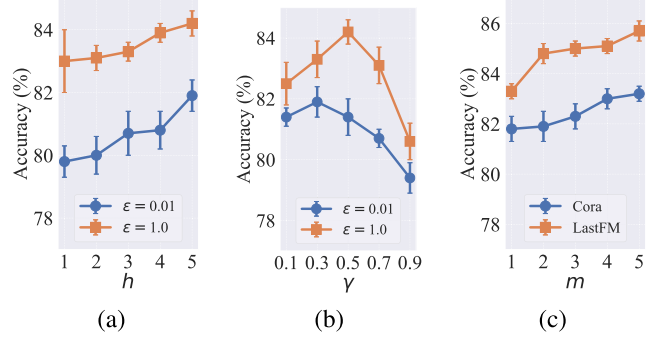


Fig. 7. (a): Accuracy of PPGNN across varied parameter h . (b): Accuracy of PPGNN across varied γ . (c): Accuracy of PPGNN across varied m .

scenarios with higher node feature variability and more complex graph structures.

E. Empirical Privacy Attack Defense

To empirically assess the privacy protection of PPGNN, we conduct attribute inference attacks (AIA) [47], [48]. In this setting, an attacker observes the perturbed features of a target node's neighbors and infers the target's sensitive attribute via majority voting, leveraging homophily in the graph. We evaluate four methods, NonPriv, BASE, LPGNN, and PPGNN, on the Cora dataset under varying privacy budgets $\epsilon \in \{0.01, 0.1, 1.0\}$. As shown in the left panel of Fig. 5, NonPriv suffers from high attack accuracy ($> 90\%$), indicating severe privacy leakage. In contrast, all LDP-based methods achieve significantly lower attack accuracy, approaching random guessing, validating their strong defense against attribute inference. Notably, PPGNN exhibits similar protection to other LDP methods, as expected under the same privacy guarantees. Its key advantage lies in preserving comparable privacy while achieving superior utility, as demonstrated in earlier sections.

F. Parameter Analysis

1) *On the Effects of h :* Fig. 7(a) displays the accuracy of PPGNN based on the GraphSAGE as the GNN backbone model and the Cora dataset, considering privacy budget $\epsilon \in \{0.01, 1.0\}$ and privacy level parameter $h \in \{1, 2, 3, 4, 5\}$. From Fig. 7(a),

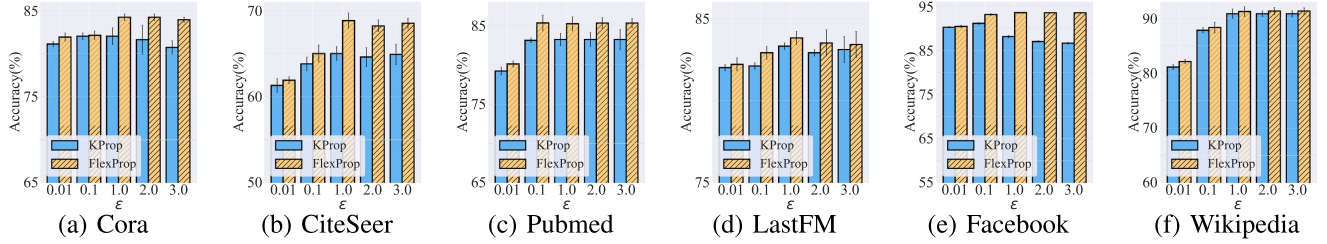


Fig. 8. Effect of FlexProp vs. KProp on graph learning performance across various datasets using the GraphSAGE model.

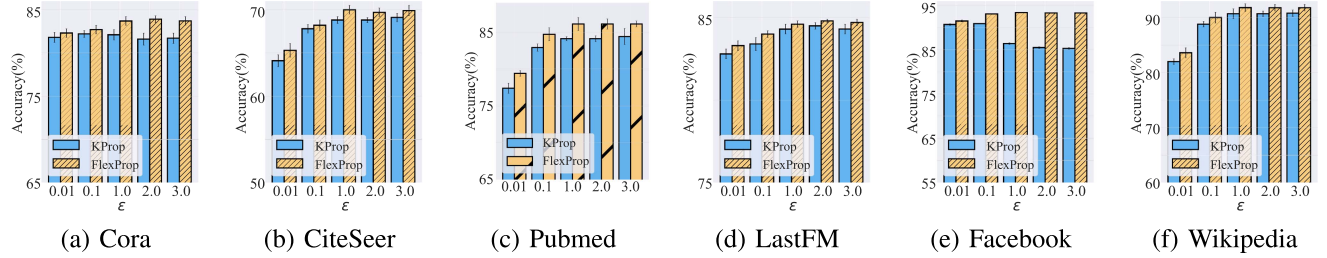


Fig. 9. Effect of FlexProp vs. KProp on graph learning performance across various datasets using the GCN model.

it is evident that as h increases, the accuracy of PPGNN consistently improves for both $\epsilon = 0.01$ and $\epsilon = 1.0$. This trend occurs because as h grows, users with lower privacy requirements are allocated a larger privacy budget, contributing to the enhancement of PPGNN's accuracy.

2) *On the Effects of γ* : Fig. 7(b) illustrates the accuracy of PPGNN based on the GraphSAGE and the Cora dataset, considering privacy budget allocation parameter $\gamma \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$ and privacy budget $\epsilon \in \{0.01, 1.0\}$. The results reveal a non-monotonic trend: model accuracy initially increases with γ , reaches a peak near the middle values, then decreases as γ continues to grow. This phenomenon reflects a fundamental trade-off in privacy budget allocation. When γ is too small, the privacy budget assigned to protecting user privacy levels is insufficient, causing large noise in the privacy level perturbation. This undermines the effectiveness of the server-side FlexProp noise calibration, ultimately reducing model utility. Conversely, when γ is too large, less budget remains for perturbing node features, leading to excessive distortion of node feature data and hence lower accuracy. Therefore, carefully balancing the allocation between protecting privacy levels and node features is crucial. Our experiments suggest that intermediate γ values, typically around 0.5, tend to provide the best trade-off, achieving higher utility while maintaining adequate privacy protections.

3) *On the Effects of K* : Fig. 6 displays the accuracy of PPGNN based on the GraphSAGE as the GNN backbone model, considering privacy budget $\epsilon \in \{0.01, 1.0\}$ and step parameter $K \in \{0, 2, 4, 8, 16\}$. From Fig. 6, as K increases, the node classification accuracy initially improves. This improvement is attributed to FlexProp algorithm effectively calibrating noisy node features by considering information from multi-hop neighbors. However, as K continues to increase, accuracy eventually decreases due to over-smoothing of output vectors caused by aggregating messages from too distant nodes, impacting

FlexProp's denoising accuracy. Therefore, selecting an appropriate value of K is crucial.

4) *On the Effects of m* : Fig. 7(c) shows the accuracy of PPGNN on Cora and LastFM with varying m . As m increases, the effective privacy budget grows, resulting in reduced noise per dimension and thus improved utility. Our experiments on the Cora dataset confirm that the proposed method consistently outperforms baselines across different values of m .

V. RELATED WORKS

A. Centralized DP-Based Graph Learning

In the centralized DP setting, a trusted curator has full access to the raw user data and adds noise during the training process to ensure privacy. Several works have explored graph learning under this framework. Wu et al. [49] study the edge re-identification attacks on GNNs and propose DPGCN, a centralized DP mechanism ensuring edge-level privacy. Kolluri et al. [50] propose a new neural network architecture called LPGNET for training graphs with privacy-sensitive edges. Daigavane et al. [51] extend the well-known DP-SGD [52] algorithm to GNN to achieve stronger node-level central DP. Sajadmanesh et al. [53] propose a novel differentially private GNN, called GAP, which safeguards the privacy of nodes and edges using aggregation perturbation. Sajadmanesh et al. [54] also propose a novel differentially private GNN called ProGAP, which improves the trade-off between accuracy and privacy by using a progressive training scheme. These approaches represent significant progress in centralized graph privacy, but they rely on a trusted data collector—an assumption that may not hold in many decentralized applications.

B. Local DP-Based Graph Learning

In contrast, LDP [36], [55] assumes no trusted aggregator, and each user perturbs their own data before sharing. This

provides stronger privacy but poses greater challenges for utility preservation. Sajadmanesh et al. [18] introduce LPGNN, the first LDP-based method for graph learning with private node features and labels. Lin et al. [19] propose Solitude, a decentralized LDP framework that protects both graph structure and features. Further, Pei et al. [22] consider privacy-enhanced issues in the context of local subgraphs. To achieve a trade-off between privacy-preserving graph topological relations and graph learning utility in locally private graph learning, Hidano et al. [33] proposed a method for training GNN models with a novel LDP algorithm called DPRR, which provides degree protection for edges of graph nodes. For edge preservation, Zhu et al. [32] introduced the BLINK. BLINK involves injecting noise into the adjacency lists and node degree to achieve privacy preservation for edges, respectively. However, none of these works consider the personalized privacy requirements of users, which is crucial in practical applications where users may have different privacy concerns. In contrast, our work addresses this gap by proposing PPGNN, the first locally differentially private GNN framework that enables personalized privacy protection across users while retaining utility. Our approach allows each user to independently choose a privacy level and leverages a flexible aggregation mechanism to mitigate the resulting noise.

C. Personalized Local Differential Privacy

To accommodate users with personalized privacy preferences, a number of works have extended LDP mechanisms into the personalized setting. Early studies such as [26], [27], [37] designed utility-optimized mechanisms for spatial data and histogram estimation, where each user may specify a different privacy budget. Shen et al. [56] proposed PLDP for aggregating multi-dimensional data, while Zeng et al. [57] studied mean estimation under personalized privacy. More recently, LDP-Tube [38] provides theoretical benchmarks for personalized LDP in high-dimensional spaces, and He et al. [58] focused on range queries over mobile user data under heterogeneous privacy settings. While these methods demonstrate the importance of personalization in LDP, they focus exclusively on *data collection and aggregation tasks*. In contrast, our work addresses privacy-preserving representation learning in graphs, where local perturbations propagate through network structures and interact with diverse noise scales. Our proposed FlexProp further adapts message passing to account for personalized noise during multi-hop aggregation, which has not been considered in prior PLDP work. To the best of our knowledge, this is the first personalized LDP framework designed for learning representations on graphs.

D. Privacy-Preserving Graph Learning

Beyond DP, several paradigms have been explored for privacy-preserving graph learning. Federated learning [59], [60] enables collaborative GNN training across decentralized clients without sharing raw data, but lacks formal privacy guarantees unless combined with DP. Adversarial training-based methods [61] aim to suppress sensitive attribute leakage, while graph perturbation approaches [62] obfuscate graph structure to reduce privacy

risks. In contrast, our work adopts LDP, which provides provable privacy guarantees without relying on a trusted aggregator, and further extends it to the personalized setting to accommodate heterogeneous user privacy preferences.

VI. CONCLUSION

In this paper, we introduce PPGNN (Personalized Privacy-preserving Graph Neural Network), a locally differentially private graph learning framework tailored to meet users' personalized privacy requirements. To safeguard user features while aligning with these personalized privacy preferences, we propose a novel LDP mechanism, the Personalized Perturbation Mechanism. It comprises the Multi-dimensional Local Randomizer, which protects multi-dimensional node features with utility guarantees, and the Extended Square Wave mechanism, which safeguards users' privacy levels across discrete domains effectively. Furthermore, to enhance the efficiency of noise calibration in the perturbed features, we present FlexProp, a weighted aggregation algorithm. Extensive experiments on six real-world datasets demonstrate the effectiveness and robustness of both PPGNN and its key components.

Limitations: While PPGNN demonstrates strong utility across diverse settings, we acknowledge two limitations. First, under highly heterogeneous privacy budget distributions, nodes with very low privacy budgets inject substantially more noise, and when such nodes dominate a target node's neighborhood $\mathcal{N}(v)$, the FlexProp aggregation may suffer from residual bias despite the privacy-aware weighting scheme II. Second, the framework assumes users can explicitly declare a privacy level $\tau \in \{1, \dots, h\}$, which may not always hold in practice. Developing adaptive budget calibration strategies or inferring privacy preferences from behavioral signals remains a promising direction for future work.

Applicable Scenarios: PPGNN is particularly well-suited for decentralized graph learning settings where users have heterogeneous privacy preferences, such as social platforms, mobile recommendation systems, and federated user profiling. In these contexts, a one-size-fits-all privacy budget would either over-protect low-sensitivity users (sacrificing utility) or under-protect high-sensitivity users (compromising privacy). PPGNN addresses this by enabling user-specific privacy budgets while preserving analytical utility through adaptive noise calibration.

REFERENCES

- [1] X. Wang and W. H. Wang, "Group property inference attacks against graph neural networks," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2022, pp. 2871–2884.
- [2] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 1024–1034.
- [3] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and S. Y. Philip, "A comprehensive survey on graph neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 1, pp. 4–24, Jan. 2021.
- [4] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. Int. Conf. Learn. Representations*, 2017, pp. 1–14.
- [5] M. Zhang and Y. Chen, "Link prediction based on graph neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 5171–5181.
- [6] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks," in *Proc. Int. Conf. Learn. Representations*, 2019, pp. 1–17.

- [7] T. Paul, A. Famulari, and T. Strufe, "A survey on decentralized online social networks," *Comput. Netw.*, vol. 75, pp. 437–452, 2014.
- [8] Y. Shen, X. He, Y. Han, and Y. Zhang, "Model stealing attacks against inductive graph neural networks," in *Proc. IEEE Symp. Secur. Privacy*, 2022, pp. 1175–1192.
- [9] B. Wu, X. Yang, S. Pan, and X. Yuan, "Model extraction attacks on graph neural networks: Taxonomy and realisation," in *Proc. ACM Asia Conf. Comput. Commun. Secur.*, 2022, pp. 337–350.
- [10] Z. Zhang, M. Chen, M. Backes, Y. Shen, and Y. Zhang, "Inference attacks against graph neural networks," in *Proc. 31st USENIX Secur. Symp.*, 2022, pp. 4543–4560.
- [11] Y. Zhuang et al., "Unveiling the secrets without data: Can graph neural networks be exploited through data-free model extraction attacks," in *Proc. 33rd USENIX Secur. Symp.*, 2024, pp. 1–18.
- [12] Z. Zhang, Q. Liu, Z. Huang, H. Wang, C.-K. Lee, and E. Chen, "Model inversion attacks against graph neural networks," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 9, pp. 8729–8741, Sep. 2022.
- [13] C. Dwork, F. McSherry, K. Nissim, and A. Smith, "Calibrating noise to sensitivity in private data analysis," in *Proc. Theory Cryptogr. Conf.*, 2006, pp. 265–284.
- [14] U. Erlingsson, V. Pihur, and A. Korolova, "RAPPOR: Randomized aggregatable privacy-preserving ordinal response," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2014, pp. 1054–1067.
- [15] A. G. Thakurta et al., "Learning new words," US Patent 9594741, 2017.
- [16] B. Ding, J. Kulkarni, and S. Yekhanin, "Collecting telemetry data privately," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 3571–3580.
- [17] X. Zhou and J. Tan, "Local differential privacy for Bayesian optimization," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 11152–11159.
- [18] S. Sajadmanesh and D. Gatica-Perez, "Locally private graph neural networks," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2021, pp. 2130–2145.
- [19] W. Lin, B. Li, and C. Wang, "Towards private learning on decentralized graphs with local differential privacy," *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 2936–2946, 2022.
- [20] W. Du, X. Ma, W. Dong, D. Zhang, C. Zhang, and Q. Sun, "Calibrating privacy budgets for locally private graph neural networks," in *Proc. Int. Conf. Netw. Netw. Appl.*, 2021, pp. 23–29.
- [21] B. Balle and Y.-X. Wang, "Improving the gaussian mechanism for differential privacy: Analytical calibration and optimal denoising," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 394–403.
- [22] X. Pei, X. Deng, S. Tian, J. Liu, and K. Xue, "Privacy-enhanced graph neural network for decentralized local graphs," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 1614–1629, 2024.
- [23] L. He, C. Li, P. Tang, and S. Su, "Going deeper into locally differentially private graph neural networks," in *Proc. 42nd Int. Conf. Mach. Learn.*, 2025, pp. 22548–22565.
- [24] K. P. Coopamootoo, M. Mehrnezhad, and E. Toreini, "'I feel invaded, annoyed, anxious and I may protect myself': Individuals' feelings about online tracking and their protective behaviour across gender and country," in *Proc. 31st USENIX Secur. Symp.*, 2022, pp. 287–304.
- [25] S. Englehardt and A. Narayanan, "Online tracking: A 1-million-site measurement and analysis," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2016, pp. 1388–1401.
- [26] R. Chen, H. Li, A. K. Qin, S. P. Kasiviswanathan, and H. Jin, "Private spatial data aggregation in the local setting," in *Proc. IEEE 32nd Int. Conf. Data Eng.*, 2016, pp. 289–300.
- [27] N. Yiwen, W. Yang, L. Huang, X. Xie, Z. Zhao, and S. Wang, "A utility-optimized framework for personalized private histogram estimation," *IEEE Trans. Knowl. Data Eng.*, vol. 31, no. 4, pp. 655–669, Apr. 2019.
- [28] C. Morris et al., "Weisfeiler and Leman go neural: Higher-order graph neural networks," in *Proc. AAAI Conf. Artif. Intell.*, 2019, pp. 4602–4609.
- [29] S. Abu-El-Haija et al., "Mixhop: Higher-order graph convolutional architectures via sparsified neighborhood mixing," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 21–29.
- [30] Y. Huang, Y. Zeng, Q. Wu, and L. Lü, "Higher-order graph convolutional network with flower-petals laplacians on simplicial complexes," in *Proc. AAAI Conf. Artif. Intell.*, 2024, pp. 12653–12661.
- [31] Z. Li, T. Wang, M. Lopuhaä-Zwakenberg, N. Li, and B. Škoric, "Estimating numerical distributions under local differential privacy," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2020, pp. 621–635.
- [32] X. Zhu, V. Y. Tan, and X. Xiao, "Blink: Link local differential privacy in graph neural networks via Bayesian estimation," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2023, pp. 2651–2664.
- [33] S. Hidano and T. Murakami, "Degree-preserving randomized response for graph neural networks under local differential privacy," *Trans. Data Privacy*, vol. 17, pp. 89–121, 2024.
- [34] M. Yang, T. Guo, T. Zhu, I. Tjuawinata, J. Zhao, and K.-Y. Lam, "Local differential privacy and its applications: A comprehensive survey," *Comput. Standards Interfaces*, vol. 89, 2023, Art. no. 103827.
- [35] C. Dwork et al., "The algorithmic foundations of differential privacy," *Found. Trends Theor. Comput. Sci.*, vol. 9, no. 3/4, pp. 211–407, 2014.
- [36] M. Yang, T. Guo, T. Zhu, I. Tjuawinata, J. Zhao, and K.-Y. Lam, "Local differential privacy and its applications: A comprehensive survey," *Comput. Standards Interfaces*, vol. 89, 2024, Art. no. 103827.
- [37] T. Murakami and Y. Kawamoto, "Utility-optimized local differential privacy mechanisms for distribution estimation," in *Proc. 28th USENIX Secur. Symp.*, 2019, pp. 1877–1894.
- [38] J. Duan, Q. Ye, H. Hu, and X. Sun, "Ldptube: Theoretical utility benchmark and enhancement for LDP mechanisms in high-dimensional space," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 8, pp. 3948–3962, Aug. 2024.
- [39] C. Dwork, "Differential privacy: A survey of results," in *Proc. Int. Conf. Theory Appl. Models Comput.*, 2008, pp. 1–19.
- [40] J. C. Duchi, M. I. Jordan, and M. J. Wainwright, "Minimax optimal procedures for locally private estimation," *J. Amer. Stat. Assoc.*, vol. 113, no. 521, pp. 182–201, 2018.
- [41] D. P. Bertsekas, *Constrained Optimization and Lagrange Multiplier Methods*. San Francisco, CA, USA: Academic Press, 2014.
- [42] Z. Yang, W. Cohen, and R. Salakhudinov, "Revisiting semi-supervised learning with graph embeddings," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 40–48.
- [43] B. Rozemberczki and R. Sarkar, "Characteristic functions on graphs: Birds of a feather, from statistical descriptors to parametric models," in *Proc. 29th ACM Int. Conf. Inf. Knowl. Manage.*, 2020, pp. 1325–1334.
- [44] B. Rozemberczki, C. Allen, and R. Sarkar, "Multi-scale attributed node embedding," *J. Complex Netw.*, vol. 9, no. 2, 2021, Art. no. cnab014.
- [45] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 1–15.
- [46] D. P. Kingma, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*.
- [47] Y. Chen et al., "Understanding and improving graph injection attack by promoting unnoticeability," in *Proc. Int. Conf. Learn. Representations*, 2022, pp. 1–14.
- [48] L. Meng, Y. Bai, Y. Chen, Y. Hu, W. Xu, and H. Weng, "Devil in disguise: Breaching graph neural networks privacy through infiltration," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2023, pp. 1153–1167.
- [49] F. Wu, Y. Long, C. Zhang, and B. Li, "LINKTELLER: Recovering private edges from graph neural networks via influence analysis," in *Proc. IEEE Symp. Secur. Privacy*, 2022, pp. 2005–2024.
- [50] A. Kolluri, T. Baluta, B. Hooi, and P. Saxena, "LPGNet: Link private graph networks for node classification," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2022, pp. 1813–1827.
- [51] Z. Xiang, T. Wang, and D. Wang, "Preserving node-level privacy in graph neural networks," in *Proc. IEEE Symp. Secur. Privacy*, 2024, pp. 4714–4732.
- [52] M. Abadi et al., "Deep learning with differential privacy," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2016, pp. 308–318.
- [53] S. Sajadmanesh, A. S. Shamsabadi, A. Bellet, and D. Gatica-Perez, "GAP: Differentially private graph neural networks with aggregation perturbation," in *Proc. 32nd USENIX Secur. Symp.*, 2023, pp. 3223–3240.
- [54] S. Sajadmanesh and D. Gatica-Perez, "ProGAP: Progressive graph neural networks with differential privacy guarantees," in *Proc. 17th ACM Int. Conf. Web Search Data Mining*, 2024, pp. 596–605.
- [55] X. Xiong, S. Liu, D. Li, Z. Cai, and X. Niu, "A comprehensive survey on local differential privacy," *Secur. Commun. Netw.*, vol. 2020, no. 1, 2020, Art. no. 8829523.
- [56] Z. Shen, Z. Xia, and P. Yu, "PLDP: Personalized local differential privacy for multidimensional data aggregation," *Secur. Commun. Netw.*, vol. 2021, no. 1, 2021, Art. no. 6684179.
- [57] Y. Zeng, S. Long, H. Liu, and H. Ding, "Collecting multidimensional numerical data and estimating mean with personalized local differential privacy," *Secur. Commun. Netw.*, vol. 2022, no. 1, 2022, Art. no. 9127412.
- [58] Y. He, M. Wang, X. Deng, P. Yang, Q. Xue, and L. T. Yang, "Personalized local differential privacy for multi-dimensional range queries over mobile user data," *IEEE Trans. Mobile Comput.*, vol. 24, no. 10, pp. 10330–10344, Oct. 2025.
- [59] C. Wang, B. Chen, G. Li, and H. Wang, "Automated graph neural network search under federated learning framework," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 10, pp. 9959–9972, Oct. 2023.
- [60] C. Zhang, Y. Xie, H. Bai, B. Yu, W. Li, and Y. Gao, "A survey on federated learning," *Knowl.-Based Syst.*, vol. 216, 2021, Art. no. 106775.

- [61] I.-C. Hsieh and C.-T. Li, "NetFense: Adversarial defenses against privacy attacks on neural networks for graph data," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 1, pp. 796–809, Jan. 2023.
- [62] P. Liao et al., "Information obfuscation of graph neural networks," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 6600–6610.



Longzhu He is currently working toward the PhD degree with the School of Computer Science, Beijing University of Posts and Telecommunications, Beijing, China. His research interests include data privacy and databases. He is also a reviewer of several leading journals and conferences, including *IEEE Transactions on Knowledge and Data Engineering*, *IEEE Transactions on Information Forensics and Security*, and *IEEE Transactions on Dependable and Secure Computing*.



Peng Tang received the PhD degree from the Beijing University of Posts and Telecommunications, Beijing, China, in 2019. He is currently an assistant professor with Shandong University, Qingdao, China. His research interests include data privacy and databases.



Chaozhuo Li received the PhD degree in computer software and theory from the School of Computer Science and Engineering, Beihang University, Beijing, China, in 2020. He is currently an associate researcher with the School of Cyber Science and Technology, Beijing University of Posts and Telecommunications, Beijing. He has authored or coauthored 60 papers, such as *NeurIPS*, *AAAI*, *SIGIR*, *ICDM*, and *CIKM*. His research interests include data mining and social network analysis.



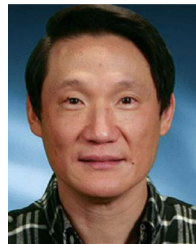
Jinhu Fu is currently working toward the PhD degree with the School of Computer Science, Beijing University of Posts and Telecommunications, Beijing, China. His research interests include graph data mining and graph learning.



Litian Zhang received the BSc degree in software engineering from Beijing Jiaotong University, Beijing, China, in 2020, and the PhD degree from the School of Cyber Science and Technology, Beihang University, Beijing, in 2025. He is currently an assistant professor with the School of Cyberspace Security, Beijing University of Posts and Telecommunications, Beijing. His research interests include social media analysis, data analysis, and data mining.



Li Sun received the PhD degree in computer science from the Beijing University of Posts and Telecommunications, Beijing, China, in 2021. He is currently an assistant professor with the Beijing University of Posts and Telecommunications, Beijing, China. He has authored or coauthored in *IEEE Transactions on Knowledge and Data Engineering*, *AAAI*, *WWW*, *IJCAI*, *ICDM*, and *CIKM*. His research interests include machine learning and data mining.



Philip S. Yu (Life Fellow, IEEE) received the PhD degree in electrical engineering from Stanford University, Stanford, CA, USA, in 1978. He is currently a distinguished professor of computer science with the Department of Computer Science, University of Illinois at Chicago, Chicago, IL, USA, and also the Wexler chair of information technology. His research interests include big data, data mining, data stream, database, and privacy. He was the recipient of the ACM SIGKDD 2016 Innovation Award, the Research Contributions Award from the IEEE International Conference on Data Mining in 2003, and Technical Achievement Award from the IEEE Computer Society in 2013. He was the editor-in-chief of *IEEE Transactions on Knowledge and Data Engineering* and *ACM Transactions on Knowledge Discovery from Data*. He is also a fellow of ACM.



Sen Su received the PhD degree in computer science from the University of Electronic Science and Technology, China, in 1998. He is currently a professor with the Beijing University of Posts and Telecommunications, Beijing, China. His research interests include data privacy and data mining.